

Planificación de Sprint (Sprint Planning)

Proyecto Educativo Gamificado — Synapxix

Proyecto:	Synapxix	Fecha del Sprint:	Junio 2026 - Semana 4
Metodología:	Ágil / Scrum	Stack Tecnológico:	Nx, Angular, NestJS, Prisma, PostgreSQL

Mensaje del Tech Lead / Project Manager:

Último sprint del mes y estamos a un paso del lanzamiento Beta. El sistema base funciona, pero ahora tiene que enganchar al usuario a largo plazo. Nos metemos de lleno en insignias, rankings y en asegurar que todo el ecosistema no se rompa bajo presión. ¡A blindar la aplicación y cerrar el mes con broche de oro!

Asignaciones y Objetivos del Equipo

Felipe

Estimación: 5 días | Prioridad: URGENTE

TÍTULO DE LA TAREA

Testing E2E y Auditoría de Seguridad (QA/Sec)

OBJETIVO PRINCIPAL

Configurar pruebas End-to-End del flujo principal y aplicar protecciones de seguridad en todo el ecosistema antes del lanzamiento.

CONTEXTO TÉCNICO

Para la beta pública no podemos dejar cabos sueltos. Deberás configurar Playwright o Cypress en el monorepo Nx y crear un script E2E (login -> jugar -> recibir feedback -> comprar en tienda). Además, implementar Rate Limiting global en NestJS y cabeceras de seguridad con Helmet.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ Suite E2E funcional que pase exitosamente simulando el flujo feliz de un alumno.
- ☐ Middlewares de Rate Limiter (ej. `ThrottlerModule`) instalados y protegiendo endpoints críticos.

- ☐ Módulo Helmet integrado y configurado para prevenir ataques XSS y configuraciones CORS restrictivas.

Sofía

Estimación: 3 días | Prioridad: **MEDIA-ALTA**

TÍTULO DE LA TAREA

Ranking de Clase / Leaderboards (Fullstack)

OBJETIVO PRINCIPAL

Implementar un sistema de clasificación para fomentar la competencia sana entre los alumnos de una misma cohorte.

CONTEXTO TÉCNICO

Las tablas SQL no son eficientes para un ranking en tiempo real con miles de puntos de experiencia modificándose constantemente. Se implementará Redis (Sorted Sets) para mantener el ranking de la clase de manera súper optimizada y se creará el componente frontend para visualizarlo.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ Configuración y conexión a Redis implementada en el backend NestJS.
- ☐ El motor de economía ahora sincroniza los puntos de experiencia con un Sorted Set en Redis de forma atómica.
- ☐ Endpoint RESTful creado para consultar el Top 10 de la clase y la posición relativa del usuario actual.

Santiago

Estimación: 4 días | Prioridad: **ALTA**

TÍTULO DE LA TAREA

Notificaciones In-App (Frontend)

OBJETIVO PRINCIPAL

Crear un sistema centralizado de notificaciones (toasts y panel) en Angular para los alumnos.

CONTEXTO TÉCNICO

El alumno debe enterarse cuando desbloquea un logro, cuando recibe un feedback del profesor o cuando sube de nivel en el ranking. Esto requiere un servicio de notificaciones global en Angular que pueda recibir eventos SSE (Server-Sent Events) o polling y mostrarlos atractivamente.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ Servicio de Angular de notificaciones con estado (Signals o RxJS BehaviorSubject) implementado.
- ☐ Componente visual tipo 'Toast' maquetado con animaciones fluidas (entrada/salida).
- ☐ Campanita de notificaciones en el header de la aplicación con contador de elementos no leídos.

Fernando Gómez

Estimación: 4 días | Prioridad: **ALTA**

TÍTULO DE LA TAREA

Motor de Logros / Achievements (Backend)

OBJETIVO PRINCIPAL

Implementar una arquitectura orientada a eventos para asignar insignias a los usuarios cuando cumplan ciertas condiciones.

CONTEXTO TÉCNICO

No queremos acoplar la lógica de logros con el motor de economía. Debes utilizar el patrón Event Emitter (`@nestjs/event-emitter`). Cuando un alumno alcance hitos (ej. "Racha de 5 días jugando", "Ganar 1000 monedas"), se emite un evento, el motor de logros lo escucha y le otorga la insignia correspondiente en PostgreSQL.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ Event Emitter instalado y configurado en el backend.
- ☐ El motor de economía y cognitivo emiten los eventos genéricos (`game.completed` , `coins.earned`).
- ☐ El listener evalúa las reglas de negocio (hardcodeadas o en DB) y persiste las insignias desbloqueadas.